SIMATS ENGINEERING

INTERNET PROGRAMMING — CSA43

1. ONLINE SYMPOSIUM PORTAL

1. Identify appropriate HTML5 semantic elements missing in the structure.

The form should be placed within suitable page structure elements. A header can contain the portal title, main can contain the primary registration content, and a section can group the symposium registration form. A label should be associated with each input so users and assistive technologies can identify the fields.

2. Modify the structure to make it standards-compliant.

The following version includes the HTML5 document declaration, language, metadata, semantic structure, labels, and a submit button. The server endpoint must accept POST requests.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Symposium Registration</title>
</head>
<body>
  <header><h1>Online Symposium Registration</h1></header>
  <main>
    <section aria-labelledby="registration-title">
      <h2 id="registration-title">Register</h2>
      <form action="/register" method="post">
        <label for="name">Name:</label>
        <input id="name" name="name" type="text" required>
        <label for="email">Email:</label>
        <input id="email" name="email" type="email" required>
        <button type="submit">Submit</button>
      </form>
    </section>
  </main>
</body>
</html>
```

3. Interpret the HTTP request generated during form submission.

When the student submits the form, the browser sends an HTTP POST request to /register. The submitted field names and values, such as name and email, are normally sent in the request body using the form's encoding type. The server uses this data to process the registration. POST does not by itself encrypt data; HTTPS is needed for transport encryption.

4. Analyze the HTTP response cycle from server to browser.

The server receives and validates the request, processes the registration, and sends an HTTP response containing a status code, headers, and possibly a response body. For example, a successful response may use 200 OK or redirect the browser with 303 See Other. The browser interprets the status and headers, then displays the confirmation page or an appropriate error message.

5. Suggest improvements for usability and accessibility.

Use visible labels, required-field indicators, clear instructions, and helpful validation messages. Keep the tab order logical, ensure keyboard access and sufficient colour contrast, and provide a mobile-friendly layout. Validate input on both the client and server, show a clear success or failure message, and use HTTPS to protect information in transit.

SIMATS ENGINEERING

INTERNET PROGRAMMING — CSA43

2. CROSS-BROWSER COMPATIBILITY ISSUE

1. Identify improper nesting and missing closing tags.

The <h1> heading and <p> paragraph are opened but not explicitly closed. The , <body>, and <html> elements are closed, but the heading and paragraph markup is incomplete. The document also lacks a doctype declaration, a language attribute, character encoding, and viewport metadata.

2. Analyze how different browsers may interpret this structure.

HTML browsers use parsing rules to recover from many markup errors, so they may automatically infer missing end tags and still display the page. However, error recovery and the resulting document tree can be confusing to maintain and may lead to inconsistent layout or accessibility behaviour. A browser may also enter quirks mode when the doctype is missing.

3. Identify missing semantic elements (e.g., header, section).

A header can identify the page heading, main can wrap the primary content, and section can group the “About us” content under a meaningful heading. A footer may be added if the page contains footer information. These elements describe the purpose of content more clearly than using generic containers alone.

4. Provide a corrected W3C-compliant HTML structure.

A corrected structure is shown below. It includes explicit closing tags and standard document metadata.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Company</title>
</head>
<body>
  <header>
    <h1>Welcome</h1>
  </header>
  <main>
    <section aria-labelledby="about-title">
      <h2 id="about-title">About Us</h2>
      <p>Learn more about our company.</p>
    </section>
  </main>
</body>
</html>
```

5. Suggest techniques to ensure cross-browser compatibility.

Use valid HTML and CSS, include the HTML5 doctype, and test pages in major browsers and different screen sizes. Prefer standards-based CSS, avoid browser-specific features unless necessary, and use feature detection or suitable fallbacks. Keep CSS consistent, use responsive design, and check the page with an HTML validator and accessibility tools.

3. FORM SUBMISSION FAILURE

1. Identify issues in the form configuration.

The button is declared as type="button", so it does not submit the form by default. The form uses GET while collecting a password; GET places submitted values in the URL query string, which is unsuitable for credentials. The inputs also lack required-field validation and associated labels.

2. Analyze why the HTTP request is not triggered.

Clicking a button with type="button" does not invoke the browser's native form-submission behaviour. Unless JavaScript is attached to the button to handle the click and send a request, the browser will not submit the form. Changing it to type="submit" allows normal form submission.

3. Interpret the role of GET method in this scenario.

GET is primarily intended to retrieve a resource. Form values are appended to the URL as query parameters, where they may be stored in browser history, server logs, and intermediary records. Therefore, GET should not be used to transmit passwords or other sensitive credentials. HTTPS encrypts traffic but does not make credentials in a URL a good design choice.

4. Propose a corrected implementation.

Use POST and a submit button, add labels and required attributes, and send the form to a server endpoint that validates the credentials. The example below demonstrates the client-side structure; authentication must still be implemented securely on the server.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Login</title>
</head>
<body>
  <main>
    <h1>Login</h1>
    <form action="/submit" method="post">
      <label for="username">Username:</label>
      <input id="username" name="username" type="text" required>
      <label for="password">Password:</label>
      <input id="password" name="password" type="password" required>
      <button type="submit">Login</button>
    </form>
  </main>
</body>
</html>
```

5. Suggest improvements for secure data transmission.

Serve the login page and endpoint over HTTPS, use POST for credential submission, and never store passwords in plain text. Validate input on the server, apply rate limiting and secure session handling, and return generic error messages that do not reveal whether a username exists. Avoid logging passwords, and use established password-hashing methods on the server.